



22. Tries - Introduction

- ♦ Trie is a Tree Data Structure.
- ♦ Trie is also called Prefix Tree or Digital Tree
- ♦ Trie stores the collection of many Strings
- ♦ Operations with Trie Data Structure.
 - a) Insert
 - b) Delete
 - c) Search
 - d) Prefix Search
 - e) Lexicographical Ordering of Words

Ex:

The image shows handwritten notes illustrating Trie operations. It is divided into four sections by vertical lines:

- Insert:** insert("jlc"), insert("ant"), insert("and"), insert("in"), insert("sin").
- Search:** search("jlc") ✓, search("and") ✓, search("am") ✗, prefixSearch("am") ✓.
- Delete:** delete("ant"), delete("in").
- Result:** jlc, ~~ant~~, ~~and~~, ~~in~~, sin.



22.1. Why Trie Data Structure

- ◆ There are other data structures, like BST's and Hash Tables etc, which are used to search for a word in collection of strings.

Then why do we need trie?

Operation	HashTable	Tries
Search	$O(1)$	$O(\text{Key-Length})$
Insert	$O(1)$	$O(\text{Key-Length})$
Delete	$O(1)$	$O(\text{Key-Length})$
Prefix Search	Not Supported	$O(\text{Prefix_len} + \text{Output-Len})$
LexicoGraphical Ordering	Not Supported	$O(\text{Prefix_len} + \text{Output-Len})$

22.2. Applications of Tries

- ◆ Following are various applications of Trie Data Structure
 - ✓ Autocomplete
 - ✓ Building Dictionaries
 - ✓ Spell checker
 - ✓ IP routing (Longest prefix matching)
 - ✓ Solving word games
 - etc



21.3. Trie Node Structure

- ♦ Trie is a rooted tree.
- ♦ Trie Node contains the following fields:
 - a) Maximum of 26 links to its children
 - b) Boolean field to represent end of key or Not..

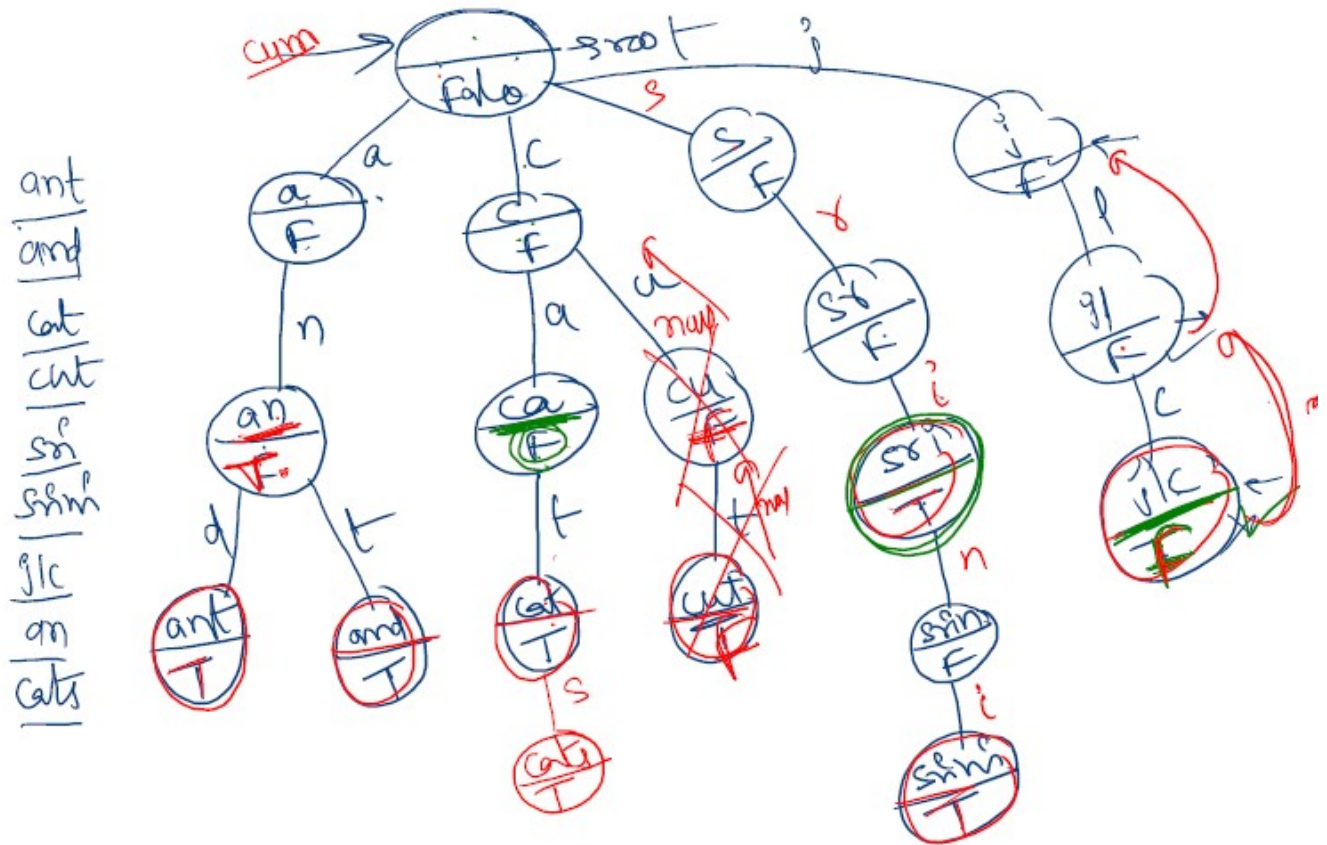
// Trie Node Structure

```
class TrieNode {  
  
    TrieNode[] children = new TrieNode[26];  
    boolean isEnd;  
  
    ....  
}
```

- ♦ Each trie consists of a root node.
- ♦ Each node of a trie represents a string and each edge represents a character.
- ♦ The root node is an empty string.
- ♦ Each node except the root node is a string having characters along the path from the root to that node.

Let us understand with an example.

Let us understand with an example.



TrieNode.java

```
package com.jlcindia.tries;
```

```
public class TrieNode {
```

```
    TrieNode children[];
    boolean isEnd;
```

```
    public TrieNode() {
        this.children= new TrieNode[26];
        this.isEnd=false;
    }
```

```
}
```



22.4. Trie - Operations

Lab1.java

```
package com.jlcindia.tries;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Trie - Operations
//Insert and Print

public class Lab1 {

    static TrieNode root;

    static void insert(String key) {
        int length = key.length();
        TrieNode currNode = root;

        for (int i = 0; i < length; i++) {
            int index = key.charAt(i) - 97;
            if (currNode.children[index] == null) {
                currNode.children[index] = new TrieNode();
            }
            currNode = currNode.children[index];
        }

        currNode.isEnd = true;
    }
}
```



```
static void printTrie(TrieNode mynode, char chArr[], int level) {

    if(mynode.isEnd) {
        System.out.println(chArr);
    }

    for(int i=0;i<26;i++) {
        if(mynode.children[i]!=null) {
            chArr[level] = (char) (i+97);
            printTrie(mynode.children[i],chArr,level+1);
        }
    }
}

public static void main(String[] args) {
    root = new TrieNode();

    insert("ant");
    insert("and");
    insert("cat");
    insert("cut");
    insert("sri");
    insert("srini");
    insert("jlc");

    int level=0;
    char chArr[] = new char[10];
    printTrie(root,chArr,level);
}
}
```



Lab2.java

```
package com.jlcindia.tries;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Trie - Operations
//Search and Prefix-Search
public class Lab2 {
    static TrieNode root;
    static void insert(String key) {
        int length = key.length();
        TrieNode currNode = root;

        for (int i = 0; i < length; i++) {
            int index = key.charAt(i) - 97;
            if (currNode.children[index] == null) {
                currNode.children[index] = new TrieNode();
            }
            currNode = currNode.children[index];
        }
        currNode.isEnd = true;
    }
    static boolean search(String key) {
        int length = key.length();
        TrieNode currNode = root;

        for (int i = 0; i < length; i++) {
            int index = key.charAt(i) - 97;
            if (currNode.children[index] == null) {
                return false;
            }
            currNode = currNode.children[index];
        }
        return currNode.isEnd ;
    }
}
```



```
static boolean prefixSearch(String key) {
    int length = key.length();
    TrieNode currNode = root;

    for (int i = 0; i < length; i++) {
        int index = key.charAt(i) - 97;
        if (currNode.children[index] == null) {
            return false;
        }
        currNode = currNode.children[index];
    }
    return true ;
}
```

```
public static void main(String[] args) {
    root = new TrieNode();

    insert("ant");
    insert("and");
    insert("cat");
    insert("cut");
    insert("srini");
    insert("jlc");

    System.out.println(search("jlc"));
    System.out.println(search("cat"));
    System.out.println(search("ca"));
    System.out.println(search("sri"));
    System.out.println(search("srini"));
    System.out.println("-----");
    System.out.println(prefixSearch("ca"));
    System.out.println(prefixSearch("sri"));
}
```



Lab3.java

```
package com.jlcindia.tries;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Trie - Operations
//Delete
public class Lab3 {
    static TrieNode root;
    static void insert(String key) {
        int length = key.length();
        TrieNode currNode = root;

        for (int i = 0; i < length; i++) {
            int index = key.charAt(i) - 97;
            if (currNode.children[index] == null) {
                currNode.children[index] = new TrieNode();
            }
            currNode = currNode.children[index];
        }
        currNode.isEnd = true;
    }
    static void printTrie(TrieNode mynode, char chArr[], int level) {

        if(mynode.isEnd) {
            System.out.println(chArr);
        }

        for(int i=0;i<26;i++) {
            if(mynode.children[i]!=null) {
                chArr[level] = (char) (i+97);
                printTrie(mynode.children[i],chArr,level+1);
            }
        }
    }
}
```



```
static boolean isEmpty(TrieNode mynode) {
    for(int i=0;i<26;i++) {
        if(mynode.children[i]!=null)
            return false;
    }
    return true;
}

static TrieNode delete(TrieNode mynode, String key, int k) {

    if(mynode==null)
        return null;

    if(k==key.length()) {
        mynode.isEnd=false;
        if(isEmpty(mynode)==true) {
            mynode=null;
        }
        return mynode;
    }

    int index = key.charAt(k) - 97;
    mynode.children[index] = delete(mynode.children[index],key,k+1);

    if(isEmpty(mynode) && mynode.isEnd==false){
        mynode=null;
    }

    return mynode;
}
```



```
public static void main(String[] args) {  
    root = new TrieNode();  
  
    insert("ant");  
    insert("and");  
    insert("an");  
    insert("cat");  
    insert("cut");  
    insert("sri");  
    insert("srini");  
    insert("jlc");  
  
    int level=0;  
    char chArr[] = new char[10];  
    printTrie(root,chArr,level);  
  
    System.out.println("-----");  
    delete(root,"jlc",0);  
    delete(root,"sri",0);  
    delete(root,"an",0);  
    delete(root,"cut",0);  
  
    level=0;  
    chArr= new char[10];  
    printTrie(root,chArr,level);  
  
    }  
}
```